

Recap — Key Concepts

A quick-reference recap of the core ideas from the deployment module: containers, AWS, and CI/CD. Each term has a one-line definition you can scan before the demo.

A. Containers & Local Dev

- **Docker** — packages an app together with all its dependencies into a portable *image* that runs the same on any machine ("works on my machine" → works everywhere).
 - **Image vs Container** — an **image** is the immutable blueprint (built once, never changes); a **container** is a running instance of that image. You can start many containers from one image.
 - **Docker Compose** — describes and runs a multi-container app from a single `docker-compose.yml`. Our local stack is one Compose file: `redis` + `api` + `web` (+ optional `voice`).
-

B. AWS

- **AWS Copilot** — a CLI that turns a Dockerfile into a full production deployment on ECS. It creates the VPC, cluster, load balancer, and roles for you so you don't wire them up by hand.
- **ECS (Elastic Container Service)** — AWS's container orchestrator. It schedules, runs, and keeps your containers alive.
- **Fargate** — *serverless* compute for ECS. It runs your containers without you managing any EC2 servers — you just pay per running task.
- **ECR (Elastic Container Registry)** — AWS's private Docker image registry. You **push** built images here; ECS **pulls** from here to run them.
- **ALB (Application Load Balancer)** — the public entry point. It spreads incoming HTTP traffic across your healthy tasks.
- **VPC & Subnet** — your own private network on AWS. **Subnets** carve it into zones: *public* (internet-facing) and *private* (internal only).
- **NAT Gateway** — lets tasks in a **private** subnet reach **out** (e.g. call external APIs) while stopping the internet from reaching **in**. *Outbound yes, inbound no*.
- **SSM Parameter Store** — secure storage for config and secrets, injected into tasks at runtime. Keeps secrets out of your image and out of git.
- **CloudWatch** — centralized logs and metrics for everything you run.
- **AWS Budgets** — cost guardrails. Emails you when spend crosses a threshold so there are no billing surprises.

C. CI/CD & The App Layer

- **GitHub Actions** — CI/CD automation. On a push, it builds the image, pushes it to ECR, and triggers a new deployment — no manual steps.
 - **OIDC** — lets GitHub Actions assume an AWS role using short-lived tokens, so you **never store long-lived AWS keys** in GitHub.
 - **Arq** — an async, Redis-backed task queue. It runs background jobs off the request path (this is our `worker` service).
-

ECS — A Closer Look

- **Cluster** — the logical group of compute capacity where your tasks run.
 - **Task & Task Definition** — the **task definition** is the blueprint (which image, how much CPU/memory, env vars, ports). A **task** is one running copy of that blueprint.
 - **Service** — keeps N tasks running, replaces any that die, and registers/deregisters them with the ALB.
 - **Desired vs Running** — **desired** = how many tasks you asked for; **running** = how many are actually up. The service constantly works to make *running* match *desired* (this is how scaling to 0 or back up works).
-

ALB — A Closer Look

- **Listener** — watches a port (e.g. 80 / 443) and forwards matching requests to a target group.
- **Target Group** — the pool of tasks the ALB can route to. Membership is controlled by health checks.
- **Health Check** — the ALB regularly pings each task (e.g. `GET /healthz`). Only tasks that pass receive traffic.
- **Distribution** — incoming requests are spread evenly across all *healthy* targets.
- **Connection Draining** — when a task is being removed, the ALB lets in-flight requests finish before the task is killed, so no live connection is dropped.

Health checks in action: **API 1** → healthy → stays in the target group and keeps serving traffic. **API 2** → failing → the ALB marks it **unhealthy** and removes it from the target group, so no traffic is sent to it until it recovers.

Handy Command — Log in to ECR

Before you can push or pull images, authenticate Docker to your private registry:

```
aws ecr get-login-password --profile "nawaloka" --region "us-west-2"
\
| docker login --username AWS --password-stdin \
  "<ACCOUNT_ID>.dkr.ecr.us-west-2.amazonaws.com"
```

This fetches a temporary ECR password and pipes it straight into `docker login`. Replace `<ACCOUNT_ID>` with your AWS account ID (and the region/profile if yours differ).

Nawaloka Hospital AI Assistant — AI Engineer Essentials · deployment module recap.